

An operating system for a small consultancy

Three connected apps — TRACKR, PAY and GROW — plus the path to put them on the team's hands.

What it is

MIS Studio is a self-hosted, three-app back office for a small consultancy or agency. All three apps run as single-file HTML applications, share the same design language, the same team roster, and the same cloud database (Supabase), and link together through a unified sidebar and single sign-on. Each app owns a distinct slice of the business but reads from and writes to the others, so the three together function as one connected system.

The three apps

TRACKR is the operational core. It handles time tracking ("My Hours"), project and client management, team performance dashboards (utilisation, billable %, hours logged per member), and forward-looking revenue projections that pull from the Grow pipeline and the Pay payroll cost base. It is also the source of truth for team members, currencies, and design tokens — the other two apps read these from TRACKR.

PAY is the payroll and finance app. It manages staff records (salary history, allocation history, tax profiles, multiple legal entities), runs monthly payroll, handles extra and contract staff with separate pay runs, tracks vendor invoices, and integrates with Xero to pull transactions and reconcile outgoing payments. PAY is also where the shared Supabase config lives.

GROW is the sales pipeline. Deals move through configurable Kanban-style stages, with weighted probabilities feeding TRACKR's projections view. When a deal is marked Won it auto-seeds a project in TRACKR; project date changes in TRACKR sync back to the linked Grow deal. It also tracks introducers and commission.

How they connect

The apps are local-first: each writes to its own `localStorage` key (`mis-v1`, `payroll-v1`, `csm-v1`) and syncs to a shared Supabase `mis_workspaces` table for cross-device access and real-time updates. Cross-tab events keep the apps in sync within a single browser. Role-based access (Admin / Team Manager / Member) is centrally configured in TRACKR and enforced across all three apps. There is a unified JSON backup that bundles all three datasets, plus EmailJS-driven scheduled reports and personal reminders.

Operationalising it

The product itself is built and integrated. The next step is moving it off the local machine and onto a domain the team can reach — for example `method.mis.studio` — gated by Google Auth so only the agency's Google Workspace accounts can sign in. The good news: the apps are already wired for this. Supabase sync, a Google OAuth Client ID field, and an allowed-domain restriction are all built into the Account settings in TRACKR. The remaining work is deployment plumbing, not product:

- **Pick a host for the static files.** The three HTML files plus the shared CSS need to be served from a CDN-backed static host — Cloudflare Pages, Vercel, Netlify or similar. All three are free for this size of workload and support custom subdomains out of the box.
- **Connect Claude to a Git repo.** Push the current files to a GitHub (or GitLab) repository and give Claude access to it. Cloudflare Pages, Vercel and Netlify all auto-deploy on every push to `main` — so once the

repo is wired to the host, any change Claude commits goes live in under a minute. This replaces the current "edit file locally, reload tab" loop with a proper deploy pipeline, and gives the team version history, rollbacks, and a single audit trail for what changed when.

- **Point the subdomain.** Add a CNAME on `method.mis.studio` pointing at the host. TLS is automatic on all the major static hosts.
- **Set up the Google OAuth Client.** In Google Cloud Console, create an OAuth 2.0 Client ID, add `https://method.mis.studio` as an authorised JavaScript origin, then paste the Client ID and your Workspace domain into TRACKR → Account → Google Sign-In. Domain restriction means only `@yourdomain` accounts can get in.
- **Lock the Supabase config.** One admin connects Supabase once; the URL and anon key sync to every team member's browser on first load. Confirm row-level security and the anon key's permissions are scoped to read/write only the `mis_workspaces` rows.
- **Seed and onboard.** Import the current local dataset via the JSON backup, assign roles (Admin / Team Manager / Member), and walk the team through their first login. Members only need My Hours and Account; managers and admins get the rest.
- **Operational rhythm.** Decide who logs hours by when, who reviews utilisation each Monday, who locks payroll each month, and who owns the monthly JSON backup as the disaster-recovery story alongside Supabase.

Net effect: same apps, same data, same UX — but reachable from any browser the team logs into with their work Google account, with no local install and no data trapped on one laptop.